EPOS for RISC-V

EPOS for RISC-V

Table of Contents

EPOS for RISC-V 1
1. Setting up an EPOS Development Environment for RISC-V on LINUX 1
1.1. EPOS Sources 1
1.2. Toolchain 1
1.2.1. Linux 32 1
1.2.2. Linux 64 1
1.3. Compiling EPOS for RISC-V 1
1.4. Running a RISC-V bare metal example on QEMU 1
1.5. Configuring EPOS elements related to RISC-V 1
2. CPU Architectural Considerations 1
2.1. RISC-V 32 (RV32) 1
2.2. RISC-V 64 (RV32) 1
2.2.1. Bases e Extensões 1
2.2.2. Níveis de privilégios 2
2.2.3. CSR's 3
2.2.4. Diferenças entre RV32I e RV64I 4
2.3. Memory Model 5
2.3.1. Privilege modes 5
2.4. Virtual-Memory Systems (VMS) 6
2.4.1. SiFive-E: Sv32 6
2.4.2. Tabela de páginas do Sv32 6
2.4.3. SiFive-U: Sv39 8
2.4.4. Outras implementações: Sv48, Sv57 e Sv64 9
2.5. Configurando um VMS 9
2.5.1. Definição do campo MODE 9
2.5.2. Definição do campo PPN 10
2.5.3. SiFive-E (RV32) Memory Map 10
2.5.4. SiFive-U (RV64) Memory Map 11
2.5.5. Cache Configuration 13
2.5.6. MMU Configuration (for kernel mode) 18
2.6. Process Model 19
2.6.1. Thread Context 20
2.6.2. Thread Context Switching 20
3. I/O and Peripherals 27
3.1. Interrupt Controller 27
3.1.1. CLINT 27
3.1.2. PLIC 32
3.2. Timers 32
3.2.1. CLINT Timer 33
3.3. UART 33
3.3.1. SiFive UART 33
3.3.2. NS16500A 33
4. Multicore Control 33
4.1. Core Activation 33
4.2. Inter-processor Interrupts (IPIs) 33
4.3. Atomic Operations 33
4.3.1. Consistência de Liberação 33
4.3.2. Ordenamento das Instruções Atômicas 33

4.3.3. Instruções Load-Reserved/Store-Conditional 34
4.3.4. Sucesso Eventual das Instruções Store-Conditional 34
4.3.5. Atomic Memory Operations 35
4.4. Locking 36
4.5. Timing 37
Contributions 37
Review Log 37

EPOS for RISC-V

1. Setting up an EPOS Development Environment for RISC-V on LINUX

1.1. EPOS Sources

1.2. Toolchain

1.2.1. Linux 32

1.2.2. Linux 64

1.3. Compiling EPOS for RISC-V

1.4. Running a RISC-V bare metal example on QEMU

1.5. Configuring EPOS elements related to RISC-V

2. CPU Architectural Considerations

2.1. RISC-V 32 (RV32)

2.2. RISC-V 64 (RV32)

2.2.1. Bases e Extensões

A definição de uma ISA única no RISC V é a partir de uma base de instruções sob inteiros que contém instruções compatíveis com as primeiras versões de RISC. Já que, a arquitetura suporta diferentes tamanhos de bits com pouca modificação da ISA, se divide a ISA em quatro bases, que se diferem pelo tamanho do registrador de inteiros e consequentemente do tamanho do espaço de endereçamento destes registradores. As duas mais importantes para o nós são a RV32I, RISC V com a extensão base de inteiros e registradores de inteiros de 32 bits, e a o RV64I, RISC V com a extensão base de inteiros e registradores de inteiros de 64 bits. Ambas estas ISA’s possuem apenas as operações base das instruções com inteiros, para adicionarmos por exemplo mais instruções, relativas a multiplicação e divisão de inteiros poderíamos utilizar a extensão M, tendo a ISA RV64IM. Abaixo uma figura com as bases e extensões possíveis para RISC V. Abaixo uma tabela com as bases e extensões.

Base	Version	Status
RVWMO	2.0	**Ratified**
RV32I	**2.1**	**Ratified**
RV64I	**2.1**	**Ratified**
RV32E	*1.9*	*Draft*
RV128I	*1.7*	*Draft*
Extension	Version	Status
Zifencei	**2.0**	**Ratified**
Zicsr	**2.0**	**Ratified**
M	**2.0**	**Ratified**
A	*2.0*	Frozen
F	**2.2**	**Ratified**
D	**2.2**	**Ratified**
Q	**2.2**	**Ratified**
C	**2.0**	**Ratified**
Ztso	*0.1*	*Frozen*
Counters	*2.0*	*Draft*
L	*0.0*	*Draft*
B	*0.0*	*Draft*
J	*0.0*	*Draft*
T	*0.0*	*Draft*
P	*0.2*	*Draft*
V	*0.7*	*Draft*
N	*1.1*	*Draft*
Zam	*0.1*	*Draft*

2.2.2. Níveis de privilégios

O RISC V define três níveis de privilégio, que podem ou não ser implementados nas implementações. Estes níveis são necessários para a proteção dos diferentes componentes da stack de software, por exemplo, uma aplicação executando sob um sistema operacional, a aplicação poderá utilizar alguns CSR's, mas não todos os que o sistema operacional tem acesso. Quando um nível de privilégio tenta acessar operações de outro nível de privilégio uma interrupção acontece alertando sobre isso.

Os três níveis de privilégios do RISC V são: User que é o nível de privilégio de aplicações executando pelo usuário, Supervisor que é o nível de privilégio que é executado pelo sistema operacional e Machine que é o nível de privilégio com mais privilégios e executa de maneira segura, utilizado no setup do sistema operacional, que depois retorna ao nível Supervisor com MRET. Abaixo a tabela sobre eles e suas identificações.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

Os níveis de privilégio são definidos por hart, ou seja, uma hart pode estar executando em um privilégio enquanto outra estará executando em outra. Ao sair de níveis de privilégios utilizamos as operações MRET, SRET e URET respectivamente para voltar ao privilégio antes de estar em Machine, Supervisor e User. Registradores que podem ser acessados por cada nível tem prefixado m, s, u para

os níveis Machine, Supervisor e User, respectivamente.

2.2.3. CSR's

Control and status register (CSR) é um registrador que armazena várias informações na CPU. RISC-V define um address space separado de no máximo 4096 CSRs.

2.2.3.1. Padrão de Nomenclatura:

Prefixo “m”, Machine-level ISA
Prefixo “s”, Supervisor-level ISA
Prefixo “u”, User-level ISA

2.2.3.2. CSRs

2.2.3.2.1. ustatus

- É um CSR que monitora e controla o estado da hart atual. Contém vários bits importantes como:

2.2.3.2.1.1. uie

- permite interrupções(interrupt-enable) global,
- tem outro CSR(mie) que permite controlar mais granularmente

2.2.3.2.1.2. upie/pp

- previous interrupt-enable/previous privilege.
- Quando acontece uma trap que leva do modo y(M|U|S) para o modo x(M|U|S), xPIE é settado ao valor de xIE, e xIE é settado a 0; xPP é settado a y
- As instruções MRET, SRET ou URET são usadas para retornar de traps dos modos M, S ou U, respectivamente.
- Quando se executa essas instruções, assumindo que xPP tem o valor “y”, xIE é settado ao valor de xPIE, o modo é mudado para y, xPIE é settado a 1; e * xPP é settado para U(ou M se não suporta User-mode)
- Pode-se usar o MRET para trocar pela primeira vez a outro modo.

2.2.3.2.1.3. mhartid

- Contém o ID da hard que está executando o código, importante para multi-core(Garantido alguma hart com ID zero).

2.2.3.2.1.4. mtvec

- pra onde vai o PC, quando ocorre uma exception/interrupção, há 2 modos

- Direto: um único endereço para onde se tratará as exceptions/interrupções
- Vectored: vai para BASE + 4 * causa, permite tratar diferentemente baseado na causa

2.2.3.2.1.5. mie

- controlar interrupt-enable mais granularmente, baseado se é timer, external, software.

2.2.3.2.1.6. mtime, mtimecmp

- Importante para scheduling, mtime é um contador, mtimecmp é um registrador que se é comparado com o valor de mtime, sempre quando mtime é > mtimecmp, é gerada uma interrupção(settar mtimecmp como mtime + quantum)

2.2.3.2.1.7. mscratch

- registrador temporário pra fazer algum cálculo durante troca de contexto

2.2.3.2.1.8. mepc

- onde o processo estava antes de ocorrer a interrupção(detalhe: é o PC antes de interromper, não o PC+4)

2.2.4. Diferenças entre RV32I e RV64I

Embora tenhamos falado que em RISC-V as ISA's de base compartilham várias instruções, principalmente com a extensão base de inteiros, há algumas diferenças entre RV32I e RV64I que são importantes para o desenvolvimento de sistemas e iremos falar delas aqui.

A principal diferença é do tamanho dos tipos como podemos ver nas tabelas abaixo. Temos um problema ocasionado, por exemplo, pelo cast de um ponteiro para um inteiro, já que, em RV32I sizeof(int) == sizeof(void *), mas em RV64I sizeof(int) != sizeof(void *). Devemos nos atentar a estas modificações de tamanho de bytes dos tipos para cada modificação.

C type	Description	Bytes in RV32	Bytes in RV64
`char`	Character value/byte	1	1
`short`	Short integer	2	2
`int`	Integer	4	4
`long`	Long integer	4	8
`long long`	Long long integer	8	8
`void*`	Pointer	4	8
`float`	Single-precision float	4	4
`double`	Double-precision float	8	8
`long double`	Extended-precision float	16	16

Outra diferença é na leitura de alguns CSR's. Em RV32I alguns CSR's por serem definidos com 64 bits, temos que realizar duas leituras de dois registradores diferentes, como, por exemplo, nos

registradores de Hardware Performance Monitoring hpmcounter1 e hpmcounter1h, onde o primeiro possui os bits menos significativos dos 64 bits e o segundo os bits mais significativos. Já em RV64I não precisamos de duas leituras, já que temos registradores de 64 bits.

Por fim, algumas instruções são alteradas. Comentamos anteriormente que a maioria das instruções se preservava, mas há algumas alterações entre RV32I e RV64I quanto às instruções. Um dos principais é sobre salvar e carregar palavras da memória. Em RV32I as instruções lw e sw, salvam e carregam, respectivamente, palavras de 32 bits na memória. Elas têm a mesma funcionalidade em RV64I. Para salvar e carregar palavras de 64 bits na memória em RV64I utilizamos as instruções ld e sw, respectivamente.

Nas outras operações mais simples de inteiros as operações são alteradas com base no XLEN da ISA. Portanto, em RV32I add adiciona dois inteiros de 32 bits e o em RV64I add adiciona dois inteiros de 64 bits. Podemos operar sob inteiros de 32 bits em RV64I utilizando o sufixo w na instrução, por exemplo, addw adiciona dois inteiros de 32 bits em RV64I.

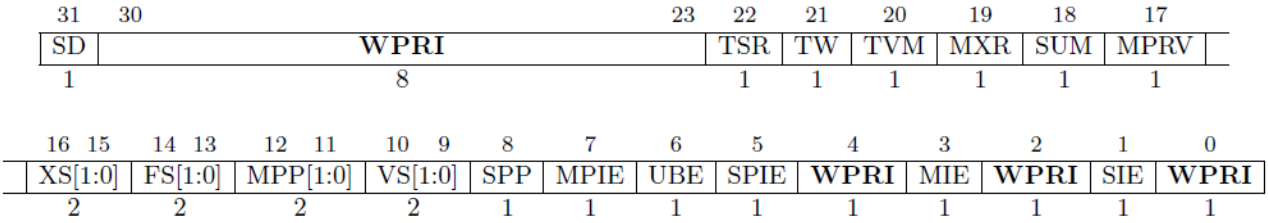
2.3. Memory Model

2.3.1. Privilege modes

A arquitetura RISC-V define 3 níveis de privilégio para execução, Machine Mode (M), Supervisor Mode (S) e User Mode (U), sendo que apenas o primeiro é considerado obrigatório para uma implementação, deixando a cargo do projetista definir se irá implementar os modos S e U.

Durante uma execução no modo M, a MMU não está ligada e portanto caso sua implementação utilize apenas esse modo, não é necessário se preocupar com os passos de configuração e habilitação da MMU.

A informação relativa ao modo de privilégio atualmente habilitado se encontra nos bits 11 e 12 do registrardor *mstatus* como exibido na figura abaixo:



Para fazer a alteração do modo, basta escrever no registrador o valor correspondente ao modo escolhido, os valores estão na figura abaixo:

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

2.4. Virtual-Memory Systems (VMS)

> Baseado no capítulo 4.1.11 do RISCV Privileged Spec

Os Virtual-Memory Systems são os sistemas que ditam como será feito a tradução de endereços virtuais para endereços físicos, o registrador *satp* (Supervisor Address Translation and Protection) é designado para configurar os valores referentes a tradução e proteção. Entre esses valores, está o campo *MODE* que seleciona o VMS atual.

2.4.1. SiFive-E: Sv32

A SiFive-E suporta 2 virtual memory systems: **Bare** e **Sv32**. O modo bare desliga a tradução de endereços, no sentido em que o endereço virtual será apenas copiado, esse modo faz com que os benefícios de utilizar memória virtual sejam ignorados. O modo **Sv32** é o que mais nos interessa para o desenvolvimento de software de sistema. Algumas características do **Sv32** são:

- Espaço de endereçamento virtual de 32 bits
- A divisão em páginas é feita com páginas de 4KiB
- Os 20 bits mais significativos representam o VPN (Virtual Page Number)
- Os 12 bits menos significativos representam o *page offset*
- A tradução de VPN para PPN é feita em uma tabela de páginas multi nível de 2 níveis
- O page offset é copiado

2.4.2. Tabela de páginas do Sv32

O **Sv32** conta com uma tabela de 2^10 entradas, cada uma de 4 bytes, sendo assim uma tabela de páginas inteira cabe dentro de uma página de 4KiB. O PPN da tabela de páginas raiz é guardado no registrador *satp* para utilização do sistema.

As figuras abaixo mostram padrões de bits relevantes para a utilização do **Sv32**. Neste caso estamos assumindo um endereçamento físico de 34 bits. Os endereços são relativamente simples, mas é importante lembrar que o VPN será extendido com zeros caso seja necessário para atingir o tamanho do PPN.

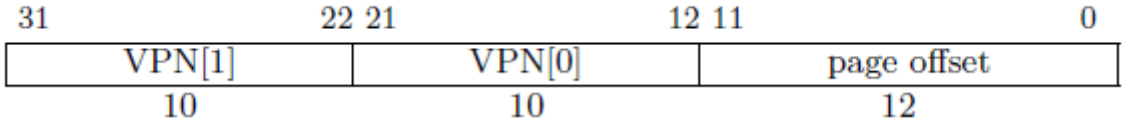


Figure 4.16: Sv32 virtual address.

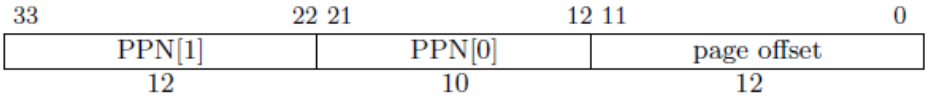


Figure 4.17: Sv32 physical address.

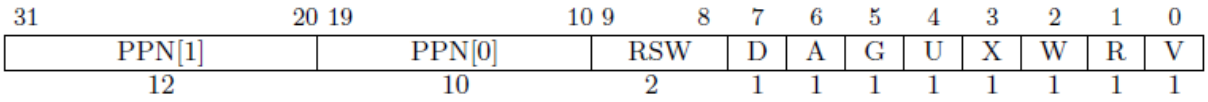


Figure 4.18: Sv32 page table entry.

A entrada da tabela de páginas possui os seguintes campos relevantes

- **Bit V**: indica a validade da entrada, quando esse bit for 0, todos os outros bits são desconsiderados
- **Bits de permissão (R, W, X)**: respectivamente indicam permissão de leitura, escrita e execução, caso os 3 bits sejam 0 significa que a entrada não é uma folha, portanto ela possui um ponteiro para a próxima entrada da tradução. Todos os padrões de valores podem ser vistos abaixo

X	W	R	Meaning
0	0	0	Pointer to next level of page table.
0	0	1	Read-only page.
0	1	0	*Reserved for future use.*
0	1	1	Read-write page.
1	0	0	Execute-only page.
1	0	1	Read-execute page.
1	1	0	*Reserved for future use.*
1	1	1	Read-write-execute page.

- **Bit U:** indica se a página pode ser acessado por software de usuário (U-mode), quando U=1 então o acesso é permitido para esse modo de privilégio
- **Bit G:** esse bit indica uma página com mapeamento global, o que significa que ela existe em todos os espaços de endereçamento. Se um nodo pai for global, todos os seus nodos filhos

também serão
- **Bit A**: indica se a página foi acessada desde a última vez que o hardware limpou os bits de acesso.
- **Bit D**: indica se a página foi escrita desde a última vez que o hardware limpou os dirty bits.
- **Bits RSW**: reservados para uso de software supervisor
- **PPN(0):** primeira parte do número físico da página
- **PPN(1):** segunda parte do número físico da página

O algoritmo de tradução de endereços virtuais está descrito no item 4.3.2 da especificação do RISC-V lançada em 2021, por ser um algoritmo relativamente longo e sem muita importância nesse contexto, não achamos necessário replicar ele aqui.

2.4.3. SiFive-U: Sv39

A SiFive-U suporta 2 virtual memory systems: **Bare** e **Sv39.** O modo bare desliga a tradução de endereços, no sentido em que o endereço virtual será apenas copiado, esse modo faz com que os benefícios de utilizar memória virtual sejam ignorados. Já o **Sv39** é baseado na implementação do **Sv32**, portanto iremos citar apenas as principais diferenças:

- O espaço de endereçamento virtual é de 39 bits
- A divisão em páginas é feita com páginas de 4KiB
- Os bits a esquerda do bit 38 devem ser complementados com o conteúdo do bit 38
- Os 27 bits mais significativos representam o VPN (Virtual Page Number)
- Os 12 bits menos significativos representam o *page offset*
- A tradução de VPN para PPN é feita em uma tabela de páginas multi nível de 3 níveis
- O page offset é copiado

Nas figuras abaixo podemos ver os padrões de bits dos endereços virtuais e físicos, além do formato de entrada na tabela de páginas. Os bits 0 a 9 tem a mesma semântica do **Sv32**, e os bits 10 a 53 contém o PPN. Os outros bits são reservados para extensões ou implementações futuras.

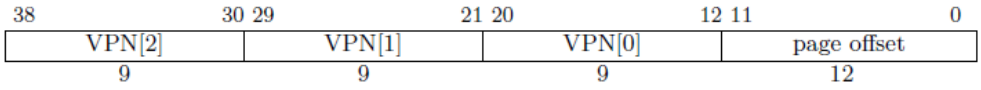


Figure 4.19: Sv39 virtual address.

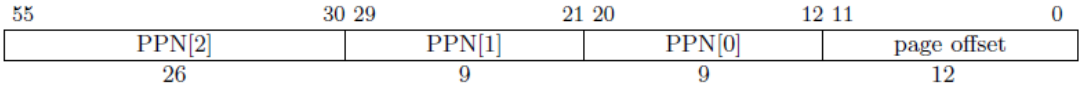


Figure 4.20: Sv39 physical address.

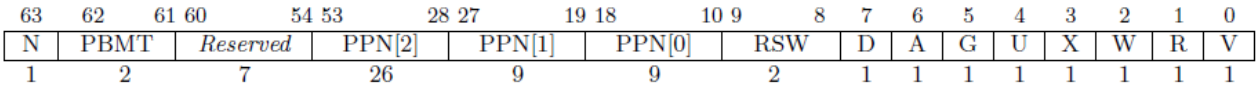


Figure 4.21: Sv39 page table entry.

O algoritmo de tradução de endereços virtuais segue o mesmo modelo do **Sv32**, inclusive sendo apenas referenciado no manual para o mesmo item 4.3.2.

2.4.4. Outras implementações: Sv48, Sv57 e Sv64

Apesar de não serem suportados pela nossa máquina target, existem outros VMSs na especificação atual do RISC-V, eles possuem características análogas aos sistemas já apresentados, apenas escalando para cima o espaço de endereçamento, a quantidade de níveis de tabelas de página.

- **Sv48:** aumenta para 4 níveis de tradução na tabela de páginas e expande o espaço de endereçamento virtual.
- **Sv57:** aumenta para 5 níveis de tradução na tabela de páginas e expande o espaço de endereçamento virtual.
- **Sv64:** na data atual, o release da especificação não detalha este VMS, apenas deixa o slot dele no registrador.

Vale citar que a especificação RISC-V comenta que implementações podem fornecer suporte a VMSs “menores” sem muito custo adicional para manter compatibilidade com software supervisor que rode apenas com esses sistemas.

2.5. Configurando um VMS

2.5.1. Definição do campo MODE

Como dito antes, o campo *MODE* do registrador *satp* para configurar qual *VMS* será utilizado. É utilizado um padrão de 4 bits, ou seja, existem espaços para implementações futuras de novos sistemas conforme necessário. Os valores definidos para as configurações se encontram na figura abaixo:

SXLEN=32		
Value	Name	Description
0	Bare	No translation or protection.
1	Sv32	Page-based 32-bit virtual addressing (see Section 4.3).

SXLEN=64		
Value	Name	Description
0	Bare	No translation or protection.
1–7	—	*Reserved for standard use*
8	Sv39	Page-based 39-bit virtual addressing (see Section 4.4).
9	Sv48	Page-based 48-bit virtual addressing (see Section 4.5).
10	Sv57	Page-based 57-bit virtual addressing (see Section 4.6).
11	*Sv64*	*Reserved for page-based 64-bit virtual addressing.*
12–13	—	*Reserved for standard use*
14–15	—	*Designated for custom use*

Table 4.4: Encoding of `satp` MODE field.

Caso você tente escrever no campo um valor reservado para o futuro ou então o valor correspondente a um *VMS* que sua implementação não aceita, a escrita completa feita no *satp* é ignorada e nenhum valor será alterado.

2.5.2. Definição do campo PPN

É necessário também definir qual será o PPN da tabela de páginas raiz, no primeiro campo do registrador *satp*, a partir dessa posição que o algoritmo de tradução de endereços virtuais será executado.

2.5.3. SiFive-E (RV32) Memory Map

Base	Top	Attr.	Description	Notes
0x0000_0000	0x0000_00FF		Reserved	Debug Address Space
0x0000_0100	0x0000_0FFF	RWX A	Debug	
0x0000_1000	0x01FF_FFFF		Reserved	
0x0200_0000	0x0200_FFFF	RW A	CLINT	On Core Complex Devices
0x0201_0000	0x07FF_FFFF		Reserved	
0x0800_0000	0x0800_1FFF	RWX A	ITIM (8 KiB)	
0x0800_2000	0x0BFF_FFFF		Reserved	
0x0C00_0000	0x0FFF_FFFF	RW A	PLIC	
0x1000_0000	0x1FFF_FFFF		Reserved	
0x2000_0000	0x3FFF_FFFF	RWX A	Peripheral Port (512 MiB)	Off Core Complex Address Space for External I/O
0x4000_0000	0x5FFF_FFFF	RWX	System Port (512 MiB)	
0x6000_0000	0x7FFF_FFFF		Reserved	
0x8000_0000	0x8000_FFFF	RWX A	Data Tightly Integrated Memory (DTIM) (64 KiB)	On Core Complex Address Space
0x8001_0000	0xFFFF_FFFF		Reserved	
0x1_0000_0000	0xF_FFFF_FFFF	RWX A	Peripheral Port (60 GiB)	
0x10_0000_0000	0xFF_FFFF_FFFF	RWX	System Port (960 GiB)	

2.5.4. SiFive-U (RV64) Memory Map

Base	Top	Attr.	Description	Notes
0x0000_0000	0x0000_00FF		Reserved	Debug Address Space
0x0000_0100	0x0000_0FFF	RWX A	Debug	
0x0000_1000	0x0000_1FFF	R X	Mode Select	On-Chip Peripherals
0x0000_2000	0x0000_FFFF		Reserved	
0x0001_0000	0x0001_7FFF	R X	Mask ROM (32 KiB)	
0x0001_8000	0x00FF_FFFF		Reserved	
0x0100_0000	0x0100_1FFF	RWX A	E51 DTIM (8 KiB)	
0x0100_2000	0x017F_FFFF		Reserved	
0x0180_0000	0x0180_1FFF	RWX A	E51 Hart 0 ITIM (8 KiB)	
0x0180_2000	0x0180_7FFF		Reserved	
0x0180_8000	0x0180_EFFF	RWX A	U54 Hart 1 ITIM (28 KiB)	
0x0180_F000	0x0180_FFFF		Reserved	
0x0181_0000	0x0181_6FFF	RWX A	U54 Hart 2 ITIM (28 KiB)	
0x0181_7000	0x0181_7FFF		Reserved	
0x0181_8000	0x0181_EFFF	RWX A	U54 Hart 3 ITIM (28 KiB)	
0x0181_F000	0x0181_FFFF		Reserved	
0x0182_0000	0x0182_6FFF	RWX A	U54 Hart 4 ITIM (28 KiB)	
0x0182_7000	0x01FF_FFFF		Reserved	
0x0200_0000	0x0200_FFFF	RW A	CLINT	
0x0201_0000	0x0201_0FFF	RW A	Cache Controller	
0x0201_1000	0x0201_FFFF		Reserved	
0x0202_0000	0x0202_0FFF	RW A	MSI	
0x0202_1000	0x02FF_FFFF		Reserved	
0x0300_0000	0x030F_FFFF	RW A	DMA Controller	
0x0310_0000	0x07FF_FFFF		Reserved	
0x0800_0000	0x09FF_FFFF	RWX A	L2 LIM (32 MiB)	
0x0A00_0000	0x0BFF_FFFF	RWXCA	L2 Zero device	
0x0C00_0000	0x0FFF_FFFF	RW A	PLIC	
0x1000_0000	0x1000_0FFF	RW A	PRCI	
0x1000_1000	0x1000_FFFF		Reserved	
0x1001_0000	0x1001_0FFF	RW A	UART 0	
0x1001_1000	0x1001_1FFF	RW A	UART 1	
0x1001_2000	0x1001_FFFF		Reserved	
0x1002_0000	0x1002_0FFF	RW A	PWM 0	
0x1002_1000	0x1002_1FFF	RW A	PWM 1	

Base	Top	Attr.	Description	Notes
0x1002_2000	0x1002_FFFF		Reserved	
0x1003_0000	0x1003_0FFF	RW A	I2C	
0x1003_1000	0x1003_FFFF		Reserved	
0x1004_0000	0x1004_0FFF	RW A	QSPI 0	
0x1004_1000	0x1004_1FFF	RW A	QSPI 1	
0x1004_2000	0x1004_FFFF		Reserved	
0x1005_0000	0x1005_0FFF	RW A	QSPI 2	
0x1005_1000	0x1005_FFFF		Reserved	
0x1006_0000	0x1006_0FFF	RW A	GPIO	
0x1006_1000	0x1006_FFFF		Reserved	
0x1007_0000	0x1007_0FFF	RW A	OTP	
0x1007_1000	0x1007_FFFF		Reserved	
0x1008_0000	0x1008_0FFF	RW A	Pin Control	
0x1008_1000	0x1008_FFFF		Reserved	
0x1009_0000	0x1009_1FFF	RW A	Ethernet MAC	
0x1009_2000	0x1009_FFFF		Reserved	
0x100A_0000	0x100A_0FFF	RW A	Ethernet Management	
0x100A_1000	0x100A_FFFF		Reserved	
0x100B_0000	0x100B_3FFF	RW A	DDR Control	
0x100B_4000	0x100B_FFFF		Reserved	
0x100C_0000	0x100C_3FFF	RW A	DDR Management	
0x100C_4000	0x17FF_FFFF		Reserved	
0x1800_0000	0x1FFF_FFFF	RW CA	Error Device	
0x2000_0000	0x2FFF_FFFF	R X A	QSPI 0 Flash (256 MiB)	Off-Chip Non-Volatile Memory
0x3000_0000	0x3FFF_FFFF	R X A	QSPI 1 Flash (256 MiB)	
0x4000_0000	0x5FFF_FFFF	RWX A	ChipLink (512 MiB)	ChipLink
0x6000_0000	0x7FFF_FFFF	RWXCA	ChipLink (512 MiB)	
0x8000_0000	0x1F_FFFF_FFFF	RWX A	DDR Memory (126 GiB)	Off-Chip Volatile Memory
0x20_0000_0000	0x2F_FFFF_FFFF	RWX A	ChipLink (64 GiB)	ChipLink
0x30_0000_0000	0x3F_FFFF_FFFF	RWXCA	ChipLink (64 GiB)	

2.5.5. Cache Configuration

2.5.5.1. **Conceito**

Representa o nível da hierarquia de memória entre o processador e a memória principal que se beneficia da localidade espacial e temporal para armazenar dados mais utilizados pelo processador.

2.5.5.1.1. **Associações**

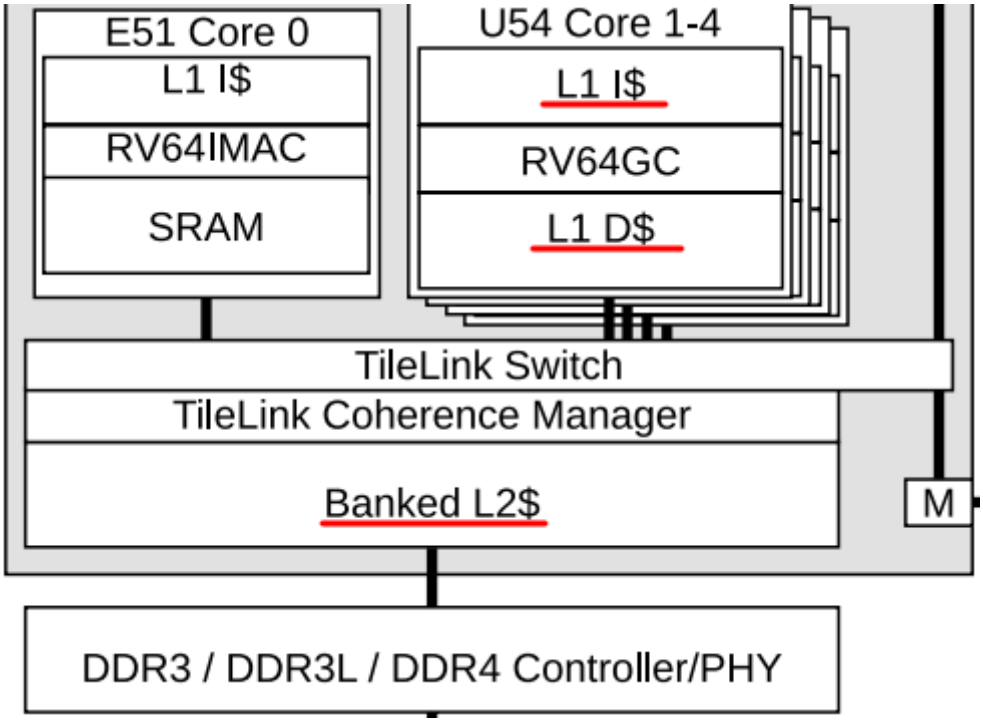
Um cache n-associativa consiste em um número de conjuntos, cada um dos quais consiste em “n” blocos. Cada bloco da memória mapeia para um conjunto único na cache fornecida pelo campo de índice onde um bloco pode ser colocado em qualquer elemento desse conjunto.

2.5.5.1.2. **Modelo de memória**

No RISC-V ISA base, a thread do hardware do RISC-V observa suas próprias operações de memória como se elas executassem sequencialmente na ordem do programa. O RISC-V possui um modelo de memória relaxado entre harts, exigindo uma instrução FENCE explícita para garantir a ordenação entre operações de memória de diferentes harts RISCV.

A instrução FENCE é usada para ordenar acessos à memória e E/S do dispositivo conforme visualizado por outros harts RISCV e dispositivos externos ou coprocessadores. Qualquer combinação de entrada do dispositivo (I), saída do dispositivo (O), leituras de memória (R) e gravações de memória (W) podem ser ordenadas em relação a qualquer combinação do mesmo. Informalmente, nenhum outro hart RISC-V ou dispositivo externo pode observar qualquer operação no conjunto sucessor após um FENCE antes de qualquer operação no conjunto predecessor antes do FENCE. O ambiente de execução definirá quais operações de E/S são possíveis e, em particular, quais instruções de carregamento e armazenamento podem ser tratadas e ordenadas como operações de entrada e saída do dispositivo, respectivamente, em vez de leituras e gravações na memória. Por exemplo, dispositivos com entrada e saída com memória mapeada normalmente serão acessados com uncached loads e stores que são solicitados usando os bits de I e O em vez dos bits R e W. As extensões do conjunto de instruções também podem descrever o novo coprocessador Instruções de E/S que também serão ordenadas usando os bits de E/S em um FENCE.

2.5.5.2. **Caches no RISC-V**



O FU540 inclui um core E51 de 64 bits com as extensões RV64IMAC e quatro cores U54 de 64 bits

com as extensões RV64IMAFDC. Tanto no E51 quanto nos U54, possuem pipeline de execução em ordem de emissão única.

2.5.5.2.1. **Cache de instruções L1**

A cache de instruções consiste em 8-way set-associative de 32 KiB para os cores U54 e 8-way set-associative de 16 KiB para o core E51. A latência de acesso de todos os blocos no sistema de memória de instruções é de um ciclo de clock. A cache não é mantida coerente com o resto do sistema de memória da plataforma. A escrita na memória de instruções deve ser sincronizada com o fluxo de busca de instruções executando uma instrução FENCE.I. Ao tentar executar uma instrução de um endereço não executável resulta em uma exceção síncrona.

2.5.5.2.2. **Cache de dados L1**

A cache de dados existe somente nos cores U54, sendo de 8-way set-associative de 32 KiB, operando na política de write-back. A latência de acesso é de dois ciclos de clock para words e double words, e três ciclos de clock para quantidades menores. Acessos desalinhados não são suportados em hardware e resultam em uma exceção para suportar a emulação de software. As caches de dados são mantidas coerentes com um gerenciador de coerência de cache baseado em diretório, que reside no cache L2 externo.

2.5.5.2.3. **Instruction Tightly Integrated Memory (ITIM)**

O ITIM fornece entrega de instruções previsíveis e de alto desempenho. Buscar uma instrução do ITIM é tão rápido quanto um acerto no cache de instruções, sem possibilidade de um falta de cache. O ITIM pode armazenar dados e instruções, embora carregue e armazene de um núcleo para outro. O ITIM não é tão eficiente quanto as DTIMs. As solicitações de memória de um núcleo para o ITIM de qualquer outro núcleo não são tão eficientes quanto a memória solicitações de um núcleo para seu próprio ITIM.

A cache de instruções pode ser parcialmente reconfigurada em ITIM tanto no core E51 quanto nos cores U52, que ocupa um intervalo de endereços fixo no mapa de memória de tamanho de 8 KiB. Exceto para 1 em unidades de linhas de cache (64 bytes, 1 way), cache de instruções deve permanecer como cache de instruções.

2.5.5.2.4. **Data Tightly Integrated Memory (DTIM)**

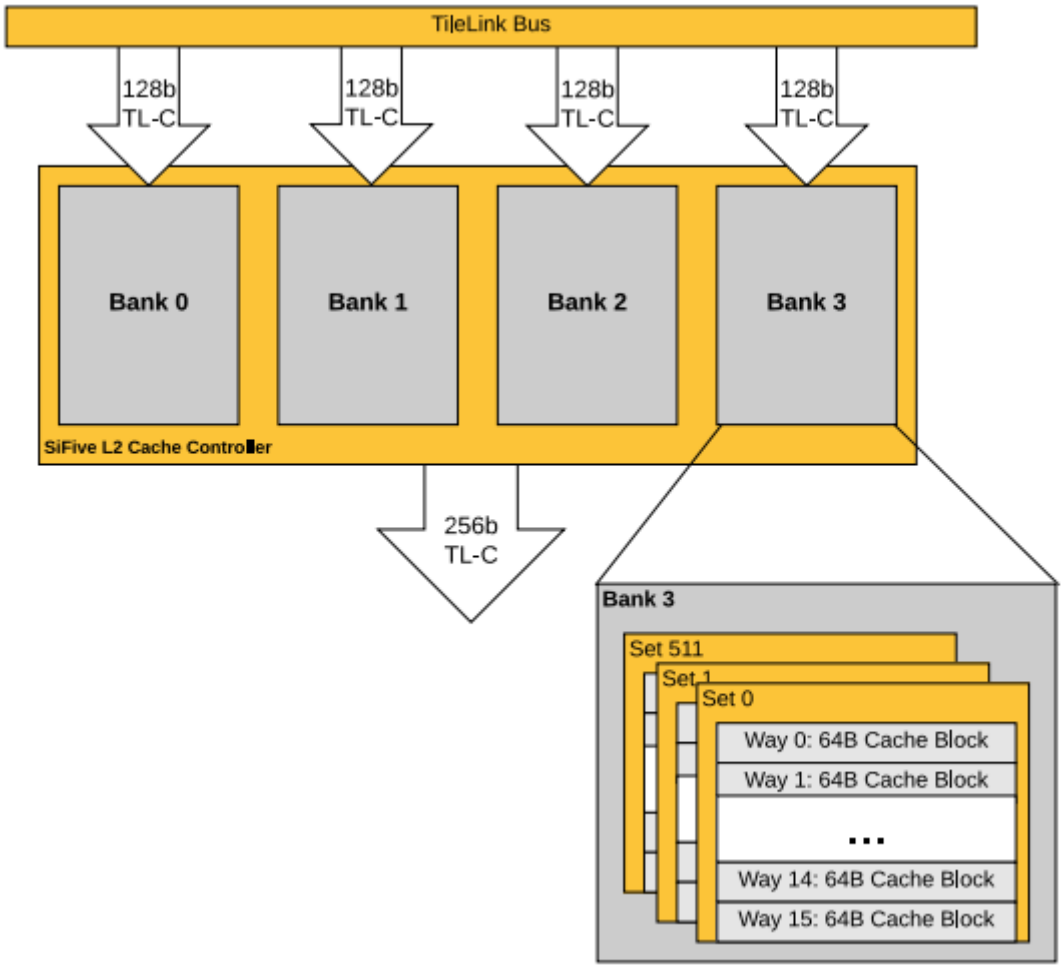
A DTIM é apenas aplicável ao core E51 e representa todo o sistema de memória de dados, possuindo capacidade de 8 KiB. A latência de acesso de um núcleo para o seu próprio DTIM é dois ciclos de clock para palavras completas e três ciclos de clock para quantidades menores. Solicitações de um núcleo de memória para DTIM de qualquer outro núcleo não são tão eficientes quanto solicitações de memória de um núcleo para seu próprio DTIM. Acessos desalinhados não são suportados em hardware e resultam em trap para permitir a emulação de software.

2.5.5.2.5. **Controlador de cache L2**

Este controlador é usado para fornecer acesso a cópias rápidas de memória para mestres em um Core Complex. O Controlador de Cache Nível 2 também atua como gerenciador de coerência baseado em diretório.

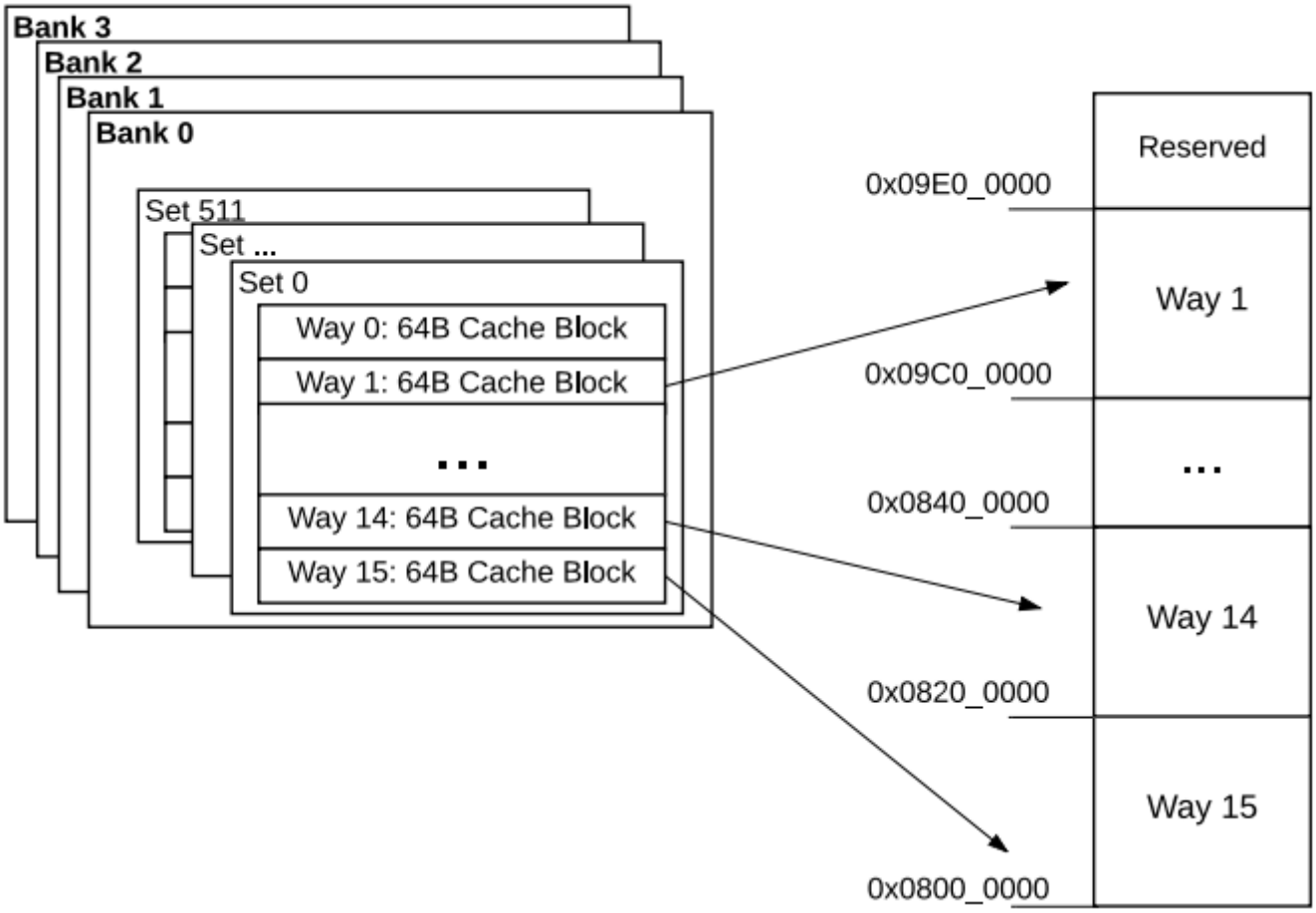
O controlador é configurado em 4 bancos. Cada banco (banks) contém 512 conjuntos (sets) de 16 ways, e cada way representa um bloco de 64 bytes.

Esta subdivisão em bancos ajuda a facilitar o aumento da largura de banda disponível entre os mestres da CPU e a cache L2, pois cada banco possui sua própria porta interna TL-C de 128 bits. Assim, várias requisições a diferentes bancos podem ocorrer em paralelo. A organização geral do controlador de cache L2 é mostrada na figura a seguir.



O controlador de cache L2 permite que suas SRAMs atuem como memória endereçada diretamente no espaço de endereço do Core Complex ou como um cache que é controlado pelo L2 Cache Controller, e que pode conter uma cópia de qualquer endereço que possa ser armazenado em cache.

Os ways da cache podem ser habilitados ou desabilitados escrevendo no registrador WayEnable. Uma vez que um way de cache é habilitado, ele não pode ser desabilitado a menos que o FU540-C000 seja reiniciado. O caminho de cache L2 de maior número é mapeado para o espaço de endereço L2-LIM mais baixo e o menor (way 1) ocupa o intervalo de endereço L2-LIM mais alto. À medida que os ways de cache L2 são habilitados, o tamanho do espaço de endereço L2-LIM diminui. A figura a seguir mostra esse mapeamento.



2.5.5.2.6. **L2 Loosely Integrated Memory (L2-LIM)**

Quando os ways de cache estão desabilitados, eles são endereçáveis no espaço de endereço L2 Loosely Integrated Memory (L2-LIM). A busca de instruções ou de dados do L2-LIM fornece um comportamento determinístico equivalente a um hit na cache L2, sem possibilidade de fault na cache. Os acessos ao L2-LIM sempre têm prioridade sobre os acessos via cache, que visam o mesmo banco de cache L2.

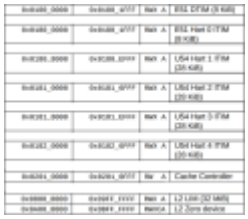
O Controlador de cache pode controlar a quantidade de memória cache que uma CPU master pode alocar usando o registrador WayMaskX. Observe que os registradores WayMaskX afetam apenas as alocações, mas as leituras ainda podem ocorrer de maneiras mascaradas. Como tal, torna-se possível bloquear formas de cache específicas, mascarando-as em todos os registradores WayMaskX. Nesse cenário, todos os mestres ainda podem ler dados nos ways de cache bloqueados, mas não podem alocar dados.

2.5.5.2.7. **L2 scratchpad**

O SiFive L2 Cache Controller possui uma região de endereço de scratchpad dedicada que permite alocação no cache usando um intervalo de endereços que não é suportado pela memória. Esta região de endereço é indicada como L2 Zero device no mapa de memória.

2.5.5.2.8. **Mapa de memória**

Nas figuras abaixo são apresentados alguns registradores para o controlador de cache L2.





2.5.5.2.9. **Registradores**

2.5.5.2.9.1. Registrador **config**

Responsável por fazer a configuração da cache L2.



2.5.5.2.9.2. Registrador **wayEnable**

Este registrador é inicializado com no reset e o valor somente pode ser aumentado.



2.5.5.2.9.3. Registrador **wayMask**

0-15: podem ser modificados.
63-16: reservados.



2.5.5.2.10. **Código de configuração para cache**

O código assembly a seguir habilita a capacidade máxima disponível para a cache L2, representado pelo valor 0x06091004. Portanto, a capacidade é: quatro bancos, 512 conjuntos, 16 ways e blocos de 64 bytes.

```
.equ CACHE_BASE,   0x02010000
.equ CACHE_INIT,   0x06091004 // 00000110 00001001 00010000 00000100
.equ CACHE_config, 0x0

li t0, CACHE_BASE
li t1, CACHE_INIT
sw t1, CACHE_config(t0)
```

2.5.6. MMU Configuration (for kernel mode)

Para configurar a MMU temos dois grandes passos, ativar o modo Supervisor e setar os valores

corretos no registrador SATP.

```
typedef unsigned long uint64;

#define MPP_MASK (3L << 11)
#define MPP_SUPERVISOR_MODE (1L << 11)

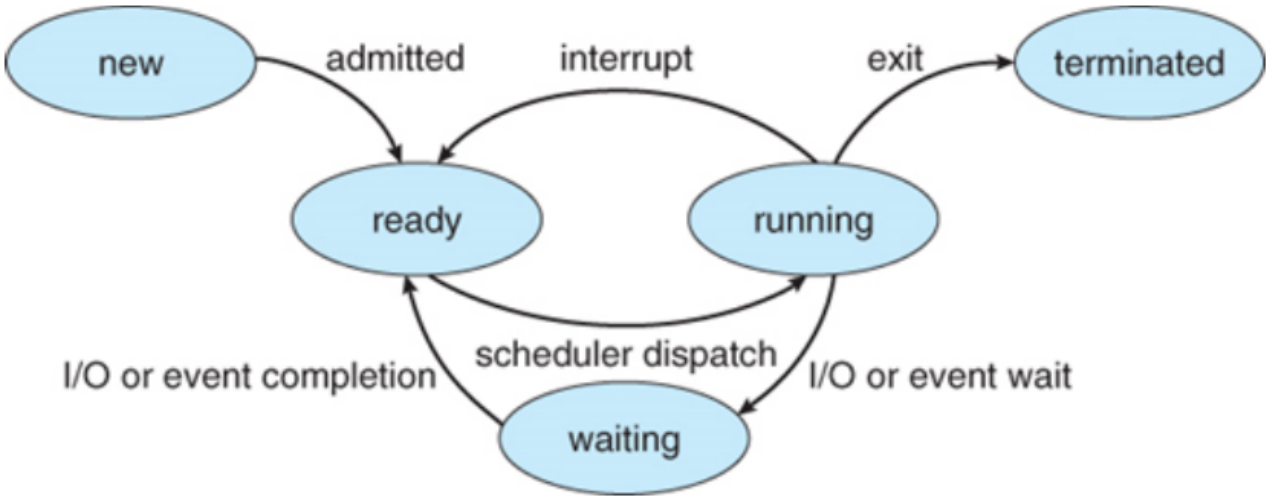
// Set M Previous Privilege mode to Supervisor, for mret.
uint64 x;
__asm__ __volatile__("csrr %0, mstatus" : "=r" (x) );
x &= ~MSTATUS_MPP_MASK;
x |= MSTATUS_MPP_S;

// Escreve o modo supervisor no MPP de mstatus
__asm__ __volatile__("csrw mstatus, %0" : : "r" (x));

uint64 heap_start = HEAP_START;
// Seta o valor de SATP
__asm__ __volatile__("csrw satp, %0" : : "r" (heap_start));
// O zero, zero significa dar um flush em todas as entradas da TLB
__asm__ __volatile__("sfence.vma zero, zero");
```

2.6. Process Model

Um processo é um programa em execução que consiste em vários elementos, como por exemplo um código do programa e um conjunto de dados. Para executar um programa, um processo deve ser criado para esse programa.
Um modelo de processo apresenta cinco fases ou estados em que um processo pode se encontrar em um determinado momento.



Os cinco estados são:

- Running: significa um processo que está sendo executado no momento.
- Ready: significa um processo que está preparado para ser executado quando for dada a oportunidade pelo sistema operacional.

- Waiting: significa que um processo não pode continuar executando até que algum evento ocorra como, por exemplo, a conclusão de uma operação

de entrada-saída.

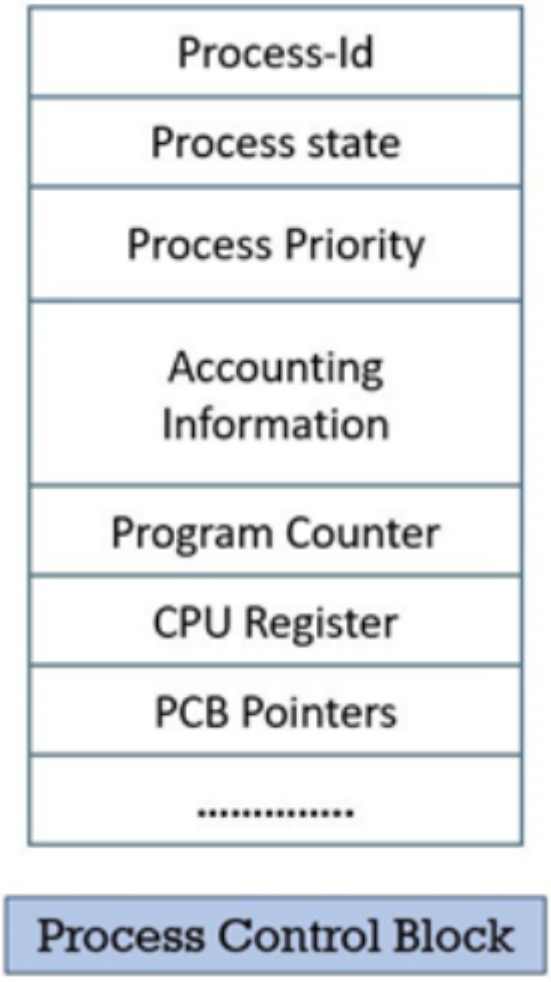
- New: significa um novo processo que foi criado, mas ainda não foi admitido pelo SO para sua execução. Um novo processo não é carregado na

memória principal, mas seu bloco de controle de processo (PCB) foi criado.

- Terminated: Um processo ou trabalho que foi liberado pelo sistema operacional, seja porque foi concluído ou foi abortado por algum problema.

2.6.1. Thread Context

O PCB (Process Control Block) é uma estrutura de dados que armazena toda informação sobre um processo. Cada processo criado possui um PCB correspondente. O PCB é importante no gerenciamento de processos, já que eles são acessados e/ou modificados pela maioria dos utilitários, principalmente aqueles relacionados ao escalonamento (troca de contexto) e no gerenciamento de recursos.



2.6.2. Thread Context Switching

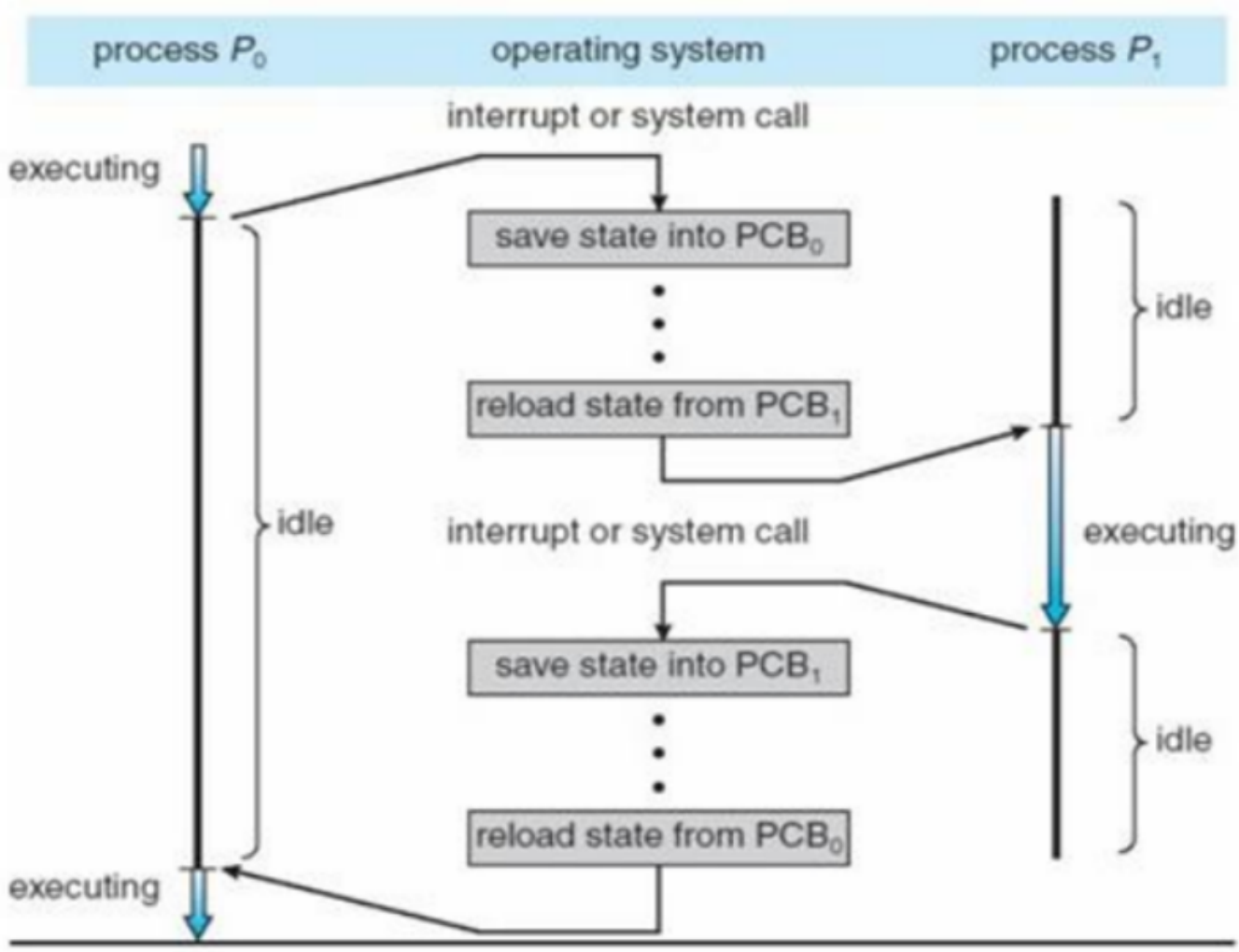
A troca de contexto basicamente é o processo que consiste em armazenar o estado de um processo

ou thread corrente, para que possa ser restaurado e retomar a execução posteriormente, permitindo que vários processos compartilhem uma única CPU, sendo um recurso essencial em um sistema operacional multitasking.
A troca de contexto pode ocorrer em três cenários:

- OS Multitasking: o escalonador seleciona um processo X com maior prioridade em relação ao processo em execução Y, realizando a troca para que X passe a executar, enquanto Y retorna ao estado ready;
- Interrupt Handling: quando ocorre uma interrupção, o hardware alterna automaticamente uma parte do contexto (pelo menos o suficiente para permitir que o manipulador retorne ao código interrompido). O manipulador pode salvar contexto adicional, dependendo dos detalhes dos projetos de hardware e software específicos. Muitas vezes, apenas uma parte mínima do contexto é alterada para minimizar o tempo gasto no tratamento da interrupção. Uma vez que o serviço de interrupção é concluído, o contexto em vigor antes da ocorrência da interrupção é restaurado para que o processo interrompido possa retomar a execução em seu estado adequado.
- User and kernel mode switching: não necessariamente é uma troca de contexto, mas dependendo do sistema operacional, uma troca de contexto também pode ocorrer neste momento.

2.6.2.1. Como ocorre a troca de contexto?

O estado do processo atualmente em execução deve ser salvo para que possa ser restaurado quando reprogramado para execução.
O estado do processo inclui todos os registradores que o processo pode estar usando, especialmente o contador de programa, além de quaisquer outros dados específicos do sistema operacional que possam ser necessários. O PCB é utilizado para armazenar esses dados.
O PCB pode ser armazenado em uma pilha por processo na memória do kernel ou pode haver alguma estrutura de dados específica definida pelo sistema operacional para essas informações. Um identificador para o PCB é adicionado a uma fila de processos que estão prontos para execução, estado ready.
Como o sistema operacional suspendeu efetivamente a execução de um processo, ele pode alternar o contexto escolhendo um processo da fila de prontos e restaurando seu PCB. Ao fazer isso, o contador de programa da PCB é carregado e, assim, a execução pode continuar no processo escolhido.



2.6.2.2. Registrador mstatus

Como dito anteriormente, o estado atual do processo em execução deve ser guardado durante a troca de contexto para que seja possível uma posterior recuperação quando necessário. O RISC-V tem um registrador responsável por guardar boa parte dessa informação chamado mstatus. Este é um registrador do tipo CSR onde guardaremos dados de controle do estado no formato booleano, em bits individuais, e outros mais específicos em campos de dois bits.
A troca de contexto pode ser custosa em termos de desempenho por conta das diversas instruções de escrita e leitura exigidas para tal. O mstatus é crucial para que possamos reduzir esse impacto, nos dando informações quanto à necessidade de guardar, ou não, determinado valor de registro.
Uma representação desse registrador com os campos a serem abordados nesta apresentação podem ser vistos na figura abaixo.

MXLEN-1	MXLEN-2 … 36	35 34	33 32	31 … 23	22	21	20	19	18	17
SD	**WPRI**	SXL[1:0]	UXL[1:0]	**WPRI**	TSR	TW	TVM	MXR	SUM	MPRV
1	MXLEN-37	2	2	9	1	1	1	1	1	1

16 15	14 13	12 11	10 9	8	7	6	5	4	3	2	1	0
XS[1:0]	FS[1:0]	MPP[1:0]	**WPRI**	SPP	MPIE	**WPRI**	SPIE	UPIE	MIE	**WPRI**	SIE	UIE
2	2	2	2	1	1	1	1	1	1	1	1	1

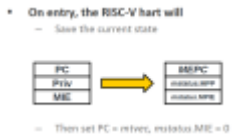
2.6.2.3. Bits □IE, □PIE e □PP do *mstatus*

São utilizados para garantir atomicidade em relação ao tratamento de interrupções dentro do modo de privilégio corrente. Os bits MIE, SIE e UIE, são usados para os modos Machine, Supervisor e User, respectivamente. Além disso, interrupções para modos de privilégios mais prioritários sempre estarão habilitados, independente do seu valor □IE. Da mesma maneira, interrupções para modos menos prioritários estarão globalmente desligadas. Vale lembrar que neste semestre, somente será usado o bit MIE, tendo em vista que somente será usado o modo de privilégio Machine.
Um ponto importante sobre os bits □IE é que como eles estão nos primeiros 5 bits do registradores, eles são facilmente setados e limpos através das instruções csrsi (set immediate) e csrci (clear immediate). Por exemplo, podemos habilitar IE no modo machine através do método mint_enable() e desabilitá-las através do método mint_disable()



Para dar suporte a interrupções encadeadas, cada modo de privilégio possui uma pilha de dois níveis dos bits interrupt-enable (IE). São os bits MPIE, SPIE e UPIE. Além disso, há também os bits MPP e SPP (UPP é implicitamente zero). Dessa maneira, quando ocorre uma interrupção do modo de usuário para modo de máquina, o bit MPIE recebe o valor de MIE, MIE é setado para zero, e MPP é setado para o modo de usuário.
Para retornar de um tratamento de interrupção, as instruções MRET, SRET e URET podem ser utilizadas para retornar do seu respectivo modo. Será usado somente a instrução MRET, neste semestre. Portanto, para se retornar do exemplo de tratamento acima, a instrução MRET iria setar o bit MIE para o valor de MPIE, o modo de privilégio é trocado para User, MPIE é setado para 1 e MPP é setado para U.

2.6.2.4. Context Switch no RISCV



Sempre que ocorrer uma interrupção no hart (núcleo), os seguintes passos são realizados automaticamente pela CPU:

1. O valor de PC é salvo no CSR mepc (Machine Exception Program Counter);
2. O modo de privilégio corrente é salvo no campo MPP do CSR mstatus;
3. O campo MIE do mstatus é salvo no campo MPIE dele mesmo;
4. PC recebe o valor do mtvec (Machine Trap Vector);
5. Por fim, é desabilitado as interrupções em modo máquina setando o campo MIE do mstatus para zero.

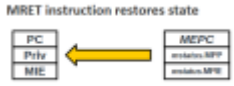
Após esses passos, a CPU passa o controle para o controlador de interrupção em software.



Essa possível implementação do controlador de interrupções implica no salvamento dos register file, especificamente os registradores x1 e x5 até x31. Além disso, é essencial saber se a exceção foi causada por uma interrupção, isso é possível lendo o último bit mais significativo do CSR mcause. E chamar o tratamento adequado, enviando os bits 62:0 (no caso do RV64) do mcause que identifica a exceção.



Após o devido tratamento da exceção, se retira da pilha os registradores previamente salvos, e finalmente é usada a instrução MRET que desfaz os passos ao se iniciar o tratamento de uma interrupção.



2.6.2.5. Estado das extensões

Um dos principais objetivos do RISC-V é dar suporte a extensões. Tendo isso em vista, alguns campos do mstatus são exclusivos para tratar esses casos. Dois exemplos são os campos FS (Floating-point State) e XS (Extension State).
Ambos são importantes pois geram um ganho de desempenho no processamento da troca de contexto permitindo que leituras e escritas sejam feitas apenas quando necessárias. Além deles, temos também o bit SD (State Dirty) que indica quando um dos dois casos teve mudança de valor e precisará ser guardado ou restaurado. Vemos na tabela a seguir as possíveis combinações de status dos campos em questão e seus respectivos significados.

Status	FS Meaning	XS Meaning
0	Off	All off
1	Initial	None dirty or clean, some on
2	Clean	None dirty, some clean
3	Dirty	Some dirty

- **Off**: Uma instrução que tente ler ou escrever o estado correspondente causará uma exceção por instrução ilegal.
- **Initial**: O estado correspondente possui valores constantes.
- **Clean**: O estado correspondente é possivelmente diferente do inicial, mas não sofreu alterações desde sua última troca de contexto.
- **Dirty**: O estado correspondente foi possivelmente alterado após sua última troca de contexto.

O contexto das extensões só precisará ser salvo caso seu status esteja definido como Dirty. Uma vez

salvo, esse status agora deve ser alterado para Clean e só volta a ser restaurado se ainda estiver nesse estado (nesse caso ele nunca deveria estar em Dirty). Caso o status seja Initial, todos os registradores devem ser passados para um valor constante pré-definido (zero, por exemplo).
As transições de contexto feitas através dos campos FS e XS devem considerar diversas possibilidades de cenário e adicionam uma maior complexidade à mudança de contexto. Elas não serão abordadas com mais profundidade neste trabalho pois nossas implementações não possuem tais extensões, forçando-as para zero (status = Off) como indicado anteriormente. As ações tomadas de acordo com cada cenário podem ser vistas na tabela abaixo:

Current State Action	Off	Initial	Clean	Dirty
At context save in privileged code				
Save state? Next state	No Off	No Initial	No Clean	Yes Clean
At context restore in privileged code				
Restore state? Next state	No Off	Yes, to initial Initial	Yes, from memory Clean	N/A N/A
Execute instruction to read state				
Action? Next state	Exception Off	Execute Initial	Execute Clean	Execute Dirty
Execute instruction to modify state, including configuration				
Action? Next state	Exception Off	Execute Dirty	Execute Dirty	Execute Dirty
Execute instruction to unconfigure unit				
Action? Next state	Exception Off	Execute Initial	Execute Initial	Execute Initial
Execute instruction to disable unit				
Action? Next state	Execute Off	Execute Off	Execute Off	Execute Off
Execute instruction to enable unit				
Action? Next state	Execute Initial	Execute Initial	Execute Initial	Execute Initial

2.6.2.6. Privilégios com o mstatus

Três níveis de privilégio são definidos no RISC-V chamados de modos. Do modo com menos privilégio para o com maior, temos: U-mode (modo de usuário / nível 0), S-mode (modo de supervisor / nível 1) e M-mode (modo de máquina / nível 3). O único modo obrigatório é o de máquina, podendo-se ter as combinações de privilégios vistas na tabela.

Supported Combinations of Modes	
Supported Levels	Modes
1	M
2	M, U
3	M, S, U

O controle de privilégios é importante pois acessos à memória em um modo podem ser protegidos de outros através da PMP (Physical Memory Protection). Isso previne que softwares que executam em modo de usuário, por exemplo, acessem endereços indesejados. Porém, nas nossas implementações utilizaremos apenas o modo de máquina.
As trocas de contexto devem considerar o modo em que os processos envolvidos estão operando, para garantir que um processo em modo de usuário retorne ao seu contexto anterior sem que haja alteração em suas permissões quanto ao sistema. Alguns campos do registrador mstatus utilizados para tal funcionalidade serão apresentados a seguir.

- **MPRV** (Modify Privilege)
  - valor 0: load e store usam a configuração do modo de privilégio atual
  - valor 1: load e store se comportam com privilégio MPP (Machine Previous Privilege)
- **SUM** (Supervisor User Memory)
  - valor 0: não é possível acessar memória de U-mode estando em S-mode
  - valor 1: o caso descrito em valor 0 se torna possível
- **MXR** (Make Executable Readable)
  - valor 0: lê apenas páginas da memória virtual marcadas como “readable” (R=1)
  - valor 1: lê páginas marcadas como “readable” e “executable” (R=1 ou X=1)

Assim como no tratamento de troca de contexto para extensões, nossa implementação permite uma complexidade menor no controle de privilégios por possuir apenas um modo.

2.6.2.7. Implementação

Para implementação foi utilizado o código do EPOS do RV32 como base. A migração para RV64 é bem simples, é necessário se atentar às instruções LW, que passaram a ser LD (Load Double Word) e os valores de incremento, no qual passaram de 4 bytes para 8 bytes.



3. I/O and Peripherals

3.1. Interrupt Controller

3.1.1. CLINT

3.1.1.1. Terminologias

- **CSRs**: Control and Status Registers.
- **HART**: hardware threads.
- **WARL**: Write Any Values, Reads Legal Values.
- **WLRL**: Write/Read Only Legal Values.
- **WPRI**: Reserved Writes Preserve Values, Reads Ignore Values.

- **Exceções** se referem a uma condição incomum no sistema ocorrida em tempo de execução em uma instrução.
    - Exemplo: O endereço de um dado que não foi alinhado corretamente em uma instrução load, faz com que a CPU entre com o tratamento de exceção do tipo "endereço desalinhado", que irá aparecer no registrador **mcause**.

- **Interrupção** se refere a um evento externo que ocorre de forma assíncrona na thread corrente. Quando uma interrupção precisa ser atendida, uma instrução é selecionada para receber a exceção de interrupção.
    - Exemplo: Um timer de interrupção é utilizado para acionar um evento futuro, sendo assim a CPU escreve em seu registrador **mtimecmp** o valor de **mtime** + ticks que se referem a um número de clocks de relógio no futuro. Como mtime se incrementa automaticamente independente de qualquer instrução executada pela CPU, em algum ponto mtimecmp se iguala a mtime e dessa forma a CPU entra com o tratador de interrupção.

- **Armadilha ou Trap**, se refere a uma transferência de controle síncrona para o tratador de armadilha devido a um condição excepcional causada na thread corrente.
    - Exemplo: Seja uma CPU com três modos de operação: Máquina, Supervisor e Usuário. Cada um deles possui seus próprios registradores de controle e status (CSRs) para tratamento de armadilha e um área de pilhada dedicada a eles. Quando em modo usuário, uma troca de contexto é requerida para tratar de um evento em modo supervisor, o software configura o sistema para uma troca de contexto e chama a instrução **ECALL** que troca o controle para o tratador de exceção de ambiente de usuário.

3.1.1.2. Registradores

3.1.1.2.1. **Machine Status (mstatus)**

- Acompanha e controla o estado operacional atual do hart.

- O bit **MIE** (Machine Interrupt Enable) indica se as interrupções estão habilitadas.
- Já o bit **MPIE** (Machine Previous Interrupt Enable) mantém o valor do bit **MIE** anterior, ou seja, salva o contexto anterior.
  - mstatus.MPIE <- mstatus.MIE
  - mstatus.MIE <- 0

- Bit **MPP** recebe **Priv** (nível de privilégio atual).
  - **mstatus.MPP = Priv**

- **MRET** instruction:
  - mstatus.MIE <- mstatus.MPIE
  - Priv <- m



3.1.1.2.2. **Machine Cause (mcause)**

- Indica qual evento que causou o trap, caso a causa seja uma interrupção o bit **Interrupt** é setado. Já o campo **Code** indica qual o código da da interrupção/exceção.



- Se for gerada mais de uma exceção síncrona, a tabela de prioridades é utilizada.



3.1.1.2.3. **Machine Exception Program Counter (mepc)**

- Quando um trap é encontrado, **mepc** recebe o endereço virtual da instrução interrompida ou que encontrou a exceção. Caso contrário, **mepc** nunca é escrito pela implementação, mas pode ser escrito explicitamente pelo software.



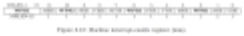
3.1.1.2.4. **Machine Interrupt Pending (mip)**

- Indica quais interrupções estão pendentes.



3.1.1.2.5. **Machine Interrupt Enable (mie)**

- Indica quais interrupções estão habilitadas.



3.1.1.2.6. **Machine Trap-Vector Base-Address (mtvec)**

- Contém o endereço base da tabela de vetores de interrupção e a configuração do modo de interrupção. Todas as exceções síncronas usam para tratamento de exceções. Sempre deve ser implementado, mas se poderá ser escrito varia com a implementação.
- Ele deve ser configurado no início do fluxo de inicialização, para eventuais tratamentos de exceções.
- O campo **BASE** consiste no endereço base da tabela de vetores e **MODE** refere-se ao modo utilizado.



3.1.1.2.7. **Machine Exception Delegation (medeleg)**

- Delega exceções ao modo supervisor.



3.1.1.2.8. **Machine Interrupt Delegation (mideleg)**

- Delega interrupções ao modo supervisor.



3.1.1.2.9. **Machine Trap Value (mtval)**

- Quando um trap é encontrado no modo Machine, **mtval** é definido como zero ou com informações específicas de exceção para auxiliar o software a lidar com o trap. Caso contrário, **mtval** nunca é escrito pela implementação, embora possa ser escrito explicitamente pelo software.



- A plataforma de hardware especificará quais exceções irão usá-lo. Quando um ponto de interrupção de hardware é acionado ou ocorre uma exceção de busca, carregamento ou armazenamento de instruções desalinhadas, acesso ou falha de página, **mtval** é gravado com o endereço virtual com falha. Seus bits mais significativos não usados, são colocados em 0.

- Opcionalmente, o registrador **mtval** também pode ser usado para retornar os bits de instrução com falha em uma exceção de instrução ilegal (**mepc** aponta para a instrução com falha na memória).

3.1.1.2.10. **Machine Software Interrupt Pending (msip)**

- Cada CPU possui seu registrador. Em sistemas com várias CPUs, uma CPU pode escrever no **msip** de qualquer outra. Isso permite uma comunicação eficiente entre processadores.
- Seu bit menos significativo é refletido no bit **mip.MSIP** e os demais estão em 0. Todos os registradores msip são zerados no reset.

3.1.1.2.11. **Machine Timer (mtime)**

- Esse registrador é único, contendo o número de ciclos a partir de **RTCCLK** (CPU real time clock) e deve ser executado em tempo constante, no reset seus campos serão zerados. Interrupções são geradas quando **mtime** >= **mtimecmp**, a qual é indicada em **mip.MTIP**, e sempre vão para o tratador modo Machine (a não ser quando delegadas ao modo Supervisor com o uso do **mideleg**).



3.1.1.2.12. **Machine Timer Compare (mtimecmp)**

- Usado em conjunto com **mtime** para interrupções de timer. Não é resetado, diferente de **mtime**. Cada CPU possui seu próprio registrador e pode ser escrito por outras CPUs.



3.1.1.3. Modos de Operação

- Existem dois modos de operação do CLINT, o **modo direto** e o **modo vetorizado**. Para configurar os modos do CLINT, escreva no campo **mtvec.MODE**, que é o bit0 do registrador de status e controle (**mtvec**):
  - Para o **modo direto**, mtvec.MODE = 0.
  - Para o **modo vetorizado**, mtvec.MODE = 1.

- O modo direto é o valor padrão de reset. O campo **mtvec.BASE** guarda o endereço base para interrupções e exceções, e deve ter um valor alinhado a um mínimo de **4 bytes no modo direto**, e um mínimo de **64 bytes no modo vetorizado**.

3.1.1.3.1. Modo Direto

- O **modo direto** significa que todas as interrupções têm armadilha para o mesmo tratador, e não há uma tabela de vetores implementada. É a responsabilidade do **software** executar código para descobrir qual interrupção ocorreu.

- O **tratador em software** no **modo direto**, deve primeiro ler **mcause.INTERRUPT** para determinar se uma interrupção ou exceção ocorreu, para então decidir o que fazer baseado no valor de **mcause.CODE** que contém o código de interrupção ou exceção respectivo.

3.1.1.3.2. Modo Vetorizado

- O modo vetorizado introduz um método para criar uma **tabela de vetores** que o **hardware**

usa para reduzir a latência do tratamento de interrupções. Quando uma interrupção acontece no **modo vetorizado**, o registrador **pc** será atribuído pelo hardware ao **endereço do índice da tabela de vetores** correspondente ao **ID da interrupção**. Do **índice da tabela de vetores**, um **jump** subsequente irá ocorrer para atender a interrupção.

- Lembre-se de que a **tabela de vetores** contém um **opcode** que é uma instrução de **jump** para um local específico.

3.1.1.4. Interrupt Levels

3.1.1.4.1. Software Interrupts - Interrupt ID #3

- Interrupções de software são acionadas ao escrever no registrador msip de certa CPU.

3.1.1.4.2. Timer Interrupts - Interrupt ID #7

- mtime >= mtimecmp

3.1.1.4.3. External Interrupts - Interrupt ID #11

- Interrupções globais geralmente vão para o PLIC primeiro e depois para CPU usando Interrupt ID #11. Em sistemas que não possuam o PLIC, pode ser desativado com mie.MEIE = 0.

3.1.1.4.4. Local Interrupts - Interrupt ID #16+

- Conexões locais podem se conectar diretamente a uma fonte de interrupção e não precisam ir para o PLIC. Sua prioridade é fixa com base em Interrupt ID. O número máximo de interrupções locais é dado por XLEN - 16.

3.1.1.5. Entrada e Saída do Tratador de Interrupções

- Sempre que ocorrer uma interrupção, o hardware salvará e restaurará automaticamente registros importantes.

```
Ao Entrar:
mepc            <- pc
mstatus.MPP     <- priv
mstatus.MPIE    <- mstatus.MIE
pc              <- mtvec (se mtvec.MODE = Direct) | mtvec.BASE + 4 * exception_code
mstatus.MIE     <- 0
```

```
Ao Sair:

pc              <- MEPC
priv            <- mstatus.MPP
mstatus.MIE     <- mstatus.MPIE
```

- **priv** refere-se ao nível de privilégio atual, o qual não é visível quando estamos operando nesse nível.

3.1.1.6. RV32 vs RV64 (no CLINT)

A única diferença é referente ao **mstatus**, onde o registrador passa de 32 para 64 bits.

3.1.2. PLIC

3.2. Timers

O RISC-V possui um contador em tempo real armazenado no registrador **mtime**. A máquina deve definir uma frequência constante como base de tempo do **mtime**. O **mtime** tem a precisão de 64 bits para os sistemas RV32, RV64 e RV128.

As plataformas fornecem o registrador de comparação, o **mtimecmp**, com 64 bits, que realiza uma interrupção quando **mtime** atingir o valor de **mtimecmp**. Esta interrupção fica registrada até que o valor de **mtimecmp** seja reescrito.

Para habilitar interrupções é usado o registrador mie (machine interrupt enable), que possui 16 bits. Nele, é possível utilizar um de seus bits com o objetivo de identificar qual tipo de interrupção será habilitada. Interrupções de timer, por exemplo, ocorrerão somente se estiverem habilitadas a partir do bit mie.**MTIE** (bit de habilitação de interrupção de timer).



Segue um exemplo de como um timer deve se comportar no RV32:

```
li a0, 0x0200BFF8
sw zero, 0(a0)
li a0, 0X02004000
addi t0, zero, 0x3
slli t0, t0, 15
sw t0, 0(a0)
sw zero, 4(a0)

la a0, trap_routine
csrw mtvec, a0
li t0, 0x00000008
csrw mstatus, t0
li t0, 0x0000001c
csrw mie, t0

loop_forever:
   li, a0, 0x0200BFF8
```

```
   lw t0, 0(a0)
j loop_forever

trap_routine:
    nop
    csrr t0, mcause
```

3.2.1. CLINT Timer

3.3. UART

3.3.1. SiFive UART

3.3.2. NS16500A

4. Multicore Control

4.1. Core Activation

4.2. Inter-processor Interrupts (IPIs)

4.3. Atomic Operations

O RISC-V possui operações atômicas que permitem ler, modificar e escrever valores na memória para dar suporte à sincronização entre os núcleos do processador rodando em um mesmo espaço de memória. Essas instruções são do tipo *load-reserved/store-conditional* e *fetch-and-op memory* e permitem que o RISC-V dê suporte ao modelo de consistência de liberação (*release consistency*).

Para arquiteturas 32 bit, as instruções são seguidas de um “W”, de **w**ord. Para arquiteturas 64 bit, as instruções são seguidas de um “D”, de **d**ouble word. Exemplo:

Arquitetura	**Exemplo de Instrução**
32 bit	LR.W
64 bit	LR.D

4.3.1. Consistência de Liberação

O modelo de consistência de liberação é um modelo relaxado (menos restritivo no contexto de sequencialismo de instruções) com o intuito de promover maior flexibilidade para utilizar paralelismo em nível de instrução, diferente de modelos como o de consistência sequencial. No modelo de consistência de liberação, a ação de entrar e sair de uma seção crítica é classificada como adquirir (*acquire*) e liberar (*release*). É esperado que exista código explícito indicando quando são realizadas essas operações.

4.3.2. Ordenamento das Instruções Atômicas

Para dar melhor suporte para a consistência de liberação, cada instrução atômica tem os bits *aq* e *rl*, utilizados para especificar restrições adicionais de ordenação. Se ambos os bits estão em zero, nenhuma restrição adicional é imposta na operação. Se somente o bit *aq* é 1, a operação atômica é tratada como um acesso de aquisição, ou seja, nenhuma operação posterior no mesmo núcleo poderá

vir antes dessa. Se somente o bit *rl* é 1, a operação atômica é tratada como um acesso de liberação, ou seja, essa operação de liberação não poderá vir antes de qualquer operação anterior a ela.

4.3.3. Instruções Load-Reserved/Store-Conditional

Para executar operações atômicas, seja de uma ou duas palavras de memória, são executadas com as instruções *load-reserved* (LR) e *store-conditional* (SC).

31 27	26	25	24 20	19 15	14 12	11 7	6 0
funct5	aq	rl	rs2	rs1	funct3	rd	opcode
5	1	1	5	5	3	5	7
LR.W/D	ordering		0	addr	width	dest	AMO
SC.W/D	ordering		src	addr	width	dest	AMO

- **Load-Reserved** carrega uma palavra do endereço em *rs1*, coloca o valor estendido de sinal em *rd* e registra um conjunto de reserva - um conjunto de bytes que inclui os bytes na palavra endereçada.
- **Store-Conditional** grava de forma condicional a palavra em *rs2* no endereço em *rs1*: a SC é bem-sucedida somente se a reserva ainda for válida e o conjunto de reservas contiver os bytes que estão sendo gravados.
    - Se a SC for bem-sucedida, é gravada a palavra em *rs2* na memória e gravada zero em *rd*.
    - Se a SC falhar, não é gravada na memória e grava um valor diferente de zero em *rd*.

Importante pontuar que, independentemente do sucesso ou falha, a execução de uma instrução *Store-Conditional* **invalida** qualquer reserva mantida por este *hart*.

4.3.4. Sucesso Eventual das Instruções Store-Conditional

A extensão padrão A define os *constrained LR/SC loops* possuem as seguintes propriedades:

- O loop compreende apenas uma sequência LR/SC e código para repetir a sequência no caso de falha, e deve conter no máximo 16 instruções colocadas sequencialmente na memória.
- Uma sequência LR/SC começa com uma instrução LR e termina com uma instrução SC. O código dinâmico executado entre as instruções LR e SC pode conter apenas instruções do conjunto de instruções base “I”, excluindo *loads*, *stores*, *backward jumps*, *taken backward branches*, instruções *JALR*, *FENCE*, *FENCE.I* e *SYSTEM*. Se a extensão “C” for suportada, então também são permitidas formas compactadas das instruções “I”.
- O código para tentar novamente uma sequência LR/SC com falha pode conter saltos e/ou ramificações para trás para repetir a sequência LR/SC, mas de outra forma tem a mesma restrição que o código entre o LR e o SC.
- Os endereços LR e SC devem estar dentro de uma região de memória com a eventualidade de LR/SC propriedade. O ambiente de execução é responsável por comunicar quais regiões possuem essa propriedade.
- O SC deve estar no mesmo endereço efetivo e do mesmo tamanho de dados que o LR mais recente executado pelo mesmo *hart*.

As sequências de LR/SC que não estão contidas nos *constrained LR/SC loops* são definidas como *unconstrained* (irrestritas). O que faz com que essas instruções irrestritas apresentem comportamento irregular, funcionando em algumas implementações e outras não.

Caso um *hart* H entre em um *constrained LR/SC loop*, o ambiente de execução deve garantir que um dos seguintes eventos eventualmente ocorra:

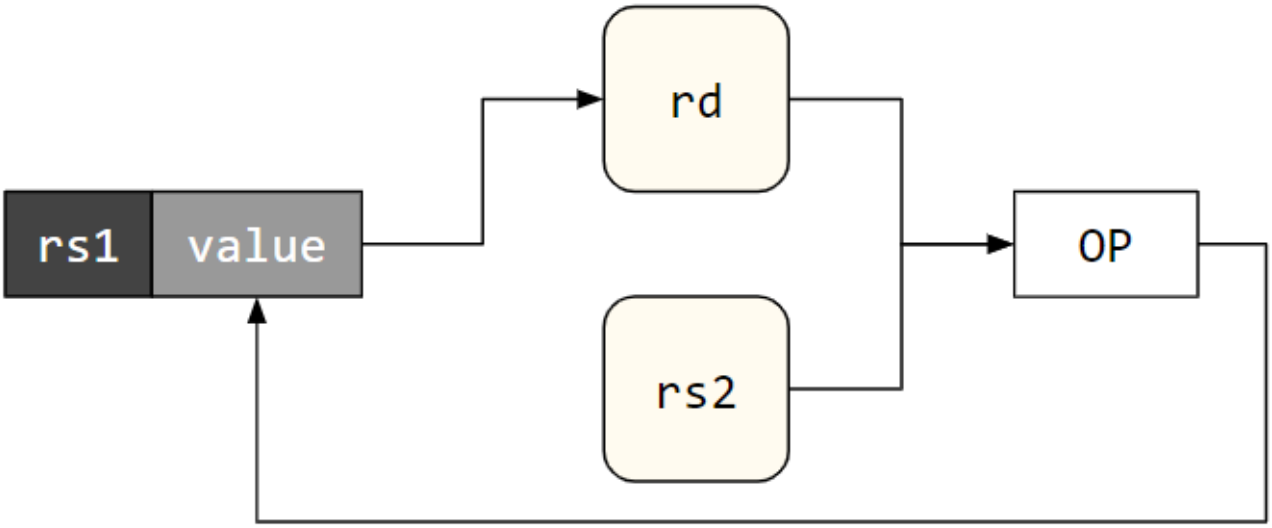
- H ou algum outro *hart* executa um SC bem-sucedido para o conjunto de reserva da instrução LR em *constrained LR/SC loop* de H.
- Algum outro *hart* executa um armazenamento incondicional ou instrução *AMO* para o conjunto de reserva de a instrução LR em *constrained LR/SC loop* de H ou algum outro dispositivo no sistema escreve a nesse conjunto de reservas.
- H executa um desvio ou salto que sai do loop LR/SC restrito.
- H fica preso.

4.3.5. Atomic Memory Operations

As operações atômicas de memória realizam operações de leitura, alteração e escrita. São formatadas como instruções do tipo R (*OP rd, rs, rt*).

31 27	26	25	24 20	19 15	14 12	11 7	6 0
funct5	aq	rl	rs2	rs1	funct3	rd	opcode
5	1	1	5	5	3	5	7
AMOSWAP.W/D	ordering		src	addr	width	dest	AMO
AMOADD.W/D	ordering		src	addr	width	dest	AMO
AMOAND.W/D	ordering		src	addr	width	dest	AMO
AMOOR.W/D	ordering		src	addr	width	dest	AMO
AMOXOR.W/D	ordering		src	addr	width	dest	AMO
AMOMAX[U].W/D	ordering		src	addr	width	dest	AMO
AMOMIN[U].W/D	ordering		src	addr	width	dest	AMO

Essas instruções carregam o valor a partir do endereço em *rs1*, coloca no registrador *rd*, aplica um operador binário no valor carregado e o valor contigo em *rs2*, e então guarda o resultado no endereço contido em *rs1*.



O endereço mantido em *rs1* deve ser naturalmente alinhado ao tamanho do operando, ou seja, 8

bytes alinhados para palavras de 64 bits e 4 bytes alinhados para palavras de 32 bits. Se o endereço não estiver alinhado naturalmente, uma exceção do tipo *address-misaligned* ou *access-fault* será gerada.

As instruções *AMO* possuem um facilitador semântico para indicar as preferências de ordenação, indicando qual o bit desejado na chamada da instrução (exemplo: *AMOSWAP.W.aq* e *AMOSWAP.W.rl*).

4.4. Locking

A arquitetura do RISC-V fornece a possibilidade de implementação de um *spin-lock* por meio da operação atômica *AMOSWAP* e outras operações básicas. Entretanto, um *spin-lock* neste nível pode custar caro, dado que são desperdiçados ciclos de CPU enquanto o lock está aguardando outra CPU, por isso é indicado a utilização apenas em pequenos trechos críticos do código.

Segue abaixo a implementação de um *spin-lock* operando sobre o endereço de memória do lock *a0*.

```
    li t0, 1 # Inicia o valor para swap.
again:
    lw t1, (a0) # verifica se o lock (em a0) está bloqueado.
    bnez t1, again # retorna se estiver bloqueado.
    amoswap.w.aq t1, t0, (a0) # Tenta adquirir o lock, colocando o valor de t0 em a0; e
a0 em t1.
    bnez t1, again # retorna se estiver bloqueado.
    # ...
    # Seção crítica.
    # ...
    amoswap.w.rl x0, x0, (a0) # Libera o lock colocando 0 em a0.
```

Para a implementação no EPOS, recomenda-se utilizar a mesma classe do utilitário de *spin-lock* já implementado (spin.h), mas adicionando a classe que implementa a operação atômica a seguir.

```
class Atomic_Spin
{
public:
    Atomic_Spin(): _locked(false) {}

    void acquire() {
        ASM("       li             t0, 1          \n");
        ASM("again: lw             t1, (%0)       \n" :: "r"(&_locked));
        ASM("       bnez           t1, again      \n");
        ASM("       amoswap.d.aq   t1, t0, (%0)   \n" :: "r"(&_locked));
        ASM("       bnez           t1, again      \n");

        db<Spin>(TRC) << "Spin::acquire[SPIN=" << this << "]()" << endl;
    }

    void release() {
```

```
        ASM("amoswap.d.rl x0, x0, (%0) \n" :: "r"(&_locked));

        db<Spin>(TRC) << "Spin::release[SPIN=" << this << "]()}" << endl;
    }

private:
    volatile bool _locked;
};
```

Ao instanciar a classe, é atribuído um endereço de memória à variável _locked, assim inicializada com 0. Com isso, ao realizar a operação de acquire, é utilizado o endereço da variável _locked na operação de swap.

4.5. Timing

Contributions

Topic	Authors	Date	Main Contributions
4.3	Gabriel Simonetti Souza	31/05/2022	Add Atomic Operations intro and AMO text
4.4	Gabriel Simonetti Souza	31/05/2022	Add Locking initial text
4.3	Gabriel Simonetti Souza	31/05/2022	Add text for Load-Reserved/Store-Conditional instructions
4.3	Gabriel Simonetti Souza	31/05/2022	Add text for Sucesso Eventual das Instruções Store-Conditional
2.4	Bryan Lima	01/06/2022	Add text for topic 2.4

Review Log

Ver	Date	Authors	Main Changes
1.0	June 1, 2022	Collaborators from INE5424@UFSC	Initial version